

Atividade Prática: Máquinas de Turing

Disciplina: Linguagens Formais e Autômatos

Instituição: Centro Universitário do Distrito Federal (UDF)

Autor: Deivid Cerqueira

RGM: 39670953

Etapa 1: Fundamentação Teórica

1. O que é uma Máquina de Turing

A Máquina de Turing (MT) é um modelo matemático abstrato de computação formalizado por Alan Turing em 1936. Trata-se de uma máquina de estados finitos acoplada a uma fita de memória de comprimento infinito, dividida em células discretas, sobre a qual opera um cabeçote de leitura e escrita capaz de mover-se bidirecionalmente (esquerda e direita). A MT estabeleceu a definição matemática formal de um dispositivo mecânico capaz de realizar qualquer cálculo computável.

2. Componentes Formais da Máquina de Turing

Formal e matematicamente, uma Máquina de Turing determinística é definida por uma 7-tupla:

$$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{accept}, q_{reject})$$

- Q : Conjunto finito e não vazio de estados do controle interno.
- Σ : Alfabeto de entrada, composto por um conjunto finito de símbolos (onde o símbolo de espaço em branco $\sqcup \notin \Sigma$).
- Γ : Alfabeto da fita, onde $\Sigma \subset \Gamma$ e $\sqcup \in \Gamma$.
- δ : Função de transição parcial, definida por $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$, indicando o próximo estado, o símbolo a ser escrito e o movimento da cabeça de leitura (L para esquerda, R para direita).
- $q_0 \in Q$: Estado inicial.
- $q_{accept} \in Q$: Estado de aceitação.
- $q_{reject} \in Q$: Estado de rejeição ($q_{accept} \neq q_{reject}$).

3. Importância Histórica e para a Ciência da Computação

A Máquina de Turing forneceu o substrato teórico para o surgimento da Ciência da Computação antes mesmo da existência de transistores ou circuitos eletrônicos. Ao demonstrar a existência da **Máquina de Turing Universal** (uma MT capaz de simular qualquer outra MT a partir do processamento de sua descrição como entrada), Turing formulou a base teórica da arquitetura dos computadores reprogramáveis modernos de programa armazenado (como a arquitetura Von Neumann). Além disso, a MT estabeleceu as fronteiras exatas do que é lógica e matematicamente computável.

4. Relação com o Conceito Formal de Algoritmo (Tese de Church-Turing)

Antes de 1936, a ideia de “algoritmo” ou “procedimento efetivo” existia apenas de forma intuitiva. A **Tese de Church-Turing** postula que o conceito informal de algoritmo equivale exatamente aos problemas passíveis de serem resolvidos por uma Máquina de Turing.

Cabe ressaltar que a Tese de Church-Turing é uma **hipótese indemonstrável formalmente** (e não um teorema matemático), pois conecta um conceito informal e intuitivo (“algoritmo”) a um formalismo matemático

estrito (MT). Até o presente, nenhum modelo computacional concebido (seja cálculo λ , funções recursivas μ , ou computação quântica) superou o poder expressivo de decisão das Máquinas de Turing.

Etapa 2: Construção da Máquina para a Linguagem $L = \{0^n 1^n \mid n \geq 1\}$

Especificação Formal

- $\Sigma = \{0, 1\}$
- $\Gamma = \{0, 1, X, Y, \sqcup\}$
- $Q = \{q_0, q_1, q_2, q_3, q_{accept}, q_{reject}\}$
- q_0 : Estado inicial
- q_{accept} : Estado de aceitação
- q_{reject} : Estado de rejeição implícito — a máquina rejeita ao travar sem transição definida para o símbolo lido, por isso ele não aparece na tabela de δ

Algoritmo de Funcionamento

- q_0 (Substituição do 0):** Lê o primeiro 0 não processado, substitui-o por X e transita para q_1 movendo à direita. Se no estado q_0 for lido Y, significa que todos os 0s já foram casados; a máquina passa para q_3 para validar o fim da fita.
- q_1 (Busca do 1 correspondente):** Move à direita ignorando símbolos 0 e Y. Ao encontrar o primeiro 1 não processado, substitui-o por Y e passa para q_2 movendo à esquerda.
- q_2 (Retorno ao início):** Move à esquerda ignorando símbolos 0 e Y até encontrar a marcação X. Ao ler X, move um passo à direita (posicionando-se no próximo 0) e retorna ao estado q_0 .
- q_3 (Validação final):** Percorre o restante da fita à direita garantindo que existem apenas símbolos Y. Se encontrar o marcador de espaço em branco ($\sqcup$), a cadeia pertence à linguagem e a máquina transita para q_{accept} .

Tabela da Função de Transição δ

Estado Atual	Símbolo Lido	Novo Estado	Símbolo Escrito	Direção
q_0	0	q_1	X	R
q_0	Y	q_3	Y	R
q_1	0	q_1	0	R
q_1	Y	q_1	Y	R
q_1	1	q_2	Y	L
q_2	0	q_2	0	L
q_2	Y	q_2	Y	L
q_2	X	q_0	X	R
q_3	Y	q_3	Y	R
q_3	$\sqcup$	q_{accept}	$\sqcup$	R

Código de Transições (Sintaxe do simulador turingmachine.io)

```
# Máquina de Turing para  $L = \{0^n 1^n \mid n \geq 1\}$ 
input: '0011'
blank: ' '
start state: q0
table:
  q0:
    '0': {write: X, R: q1}
    Y: {write: Y, R: q3}
  q1:
    '0': R
    Y: R
    '1': {write: Y, L: q2}
  q2:
    '0': L
    Y: L
    X: {R: q0}
  q3:
    Y: R
    ' ': {R: qaccept}
qaccept:
```

Etapa 3: Tabela de Simulação e Registros de Execução

Teste	Entrada	Resultado Esperado	Resultado Obtido	Estados Percorridos (Sequência Executada)
1	01	Aceito	Aceito	$q_0 \rightarrow q_1 \rightarrow q_2 \rightarrow q_0 \rightarrow q_3 \rightarrow q_{accept}$
2	0011	Aceito	Aceito	$q_0 \rightarrow q_1 \rightarrow q_1 \rightarrow q_2 \rightarrow q_2 \rightarrow q_0 \rightarrow q_1 \rightarrow q_1 \rightarrow q_2 \rightarrow q_2 \rightarrow q_0 \rightarrow q_3 \rightarrow q_3 \rightarrow q_{accept}$
3	001	Rejeitado	Rejeitado	$q_0 \rightarrow q_1 \rightarrow q_1 \rightarrow q_2 \rightarrow q_2 \rightarrow q_0 \rightarrow q_1 \rightarrow$ Travamento em q_1 (sem transição para $\sqcup$)

Etapa 4: Reflexão Sobre Limites Computacionais e Avaliação

O Problema da Parada (Halting Problem)

Formulado por Alan Turing em 1936, o Problema da Parada questiona se é possível construir um algoritmo genérico H que receba como entrada a descrição de uma Máquina de Turing qualquer M e uma cadeia de entrada w , e determine de forma genérica e em tempo finito se $M(w)$ irá parar ou rodar indefinidamente em um laço infinito.

Turing provou, por meio de uma **prova por contradição (diagonalização de Cantor)**, que tal algoritmo é **impossível**. Ao conceber uma máquina hipotética D que inverte a resposta de H aplicada a si mesma, surge um paradoxo lógico irresolúvel: D para se e somente se D não para. Logo, o Problema da Parada é formalmente **indecidível**.

Distinção Crucial: Problemas Difíceis vs. Impossíveis

1. Problemas Difíceis (Intratáveis):

São problemas **computáveis e decidíveis**, ou seja, existe algoritmo para resolvê-los de forma exata. No entanto, exigem uma quantidade de recursos de tempo ou memória computacional que cresce exponencialmente ou fatorialmente em relação ao tamanho da entrada ($O(2^n)$, $O(n!)$). Para entradas grandes, a execução torna-se inviável na prática.

Exemplo: Problema do Caixeiro Viajante (TSP) e Problema da Mochila (Knapsack).

2. Problemas Impossíveis (Indecidíveis / Não-Computáveis):

São problemas para os quais **não existe e jamais existirá** um algoritmo capaz de fornecer uma solução genérica e correta para todas as instâncias em tempo finito, independentemente de poder de processamento ou avanços de hardware. Trata-se de uma limitação lógica da própria computação.

Exemplo: Problema da Parada, Décimo Problema de Hilbert e o Problema da Correspondência de Post (PCP).

Checklist de Submissão

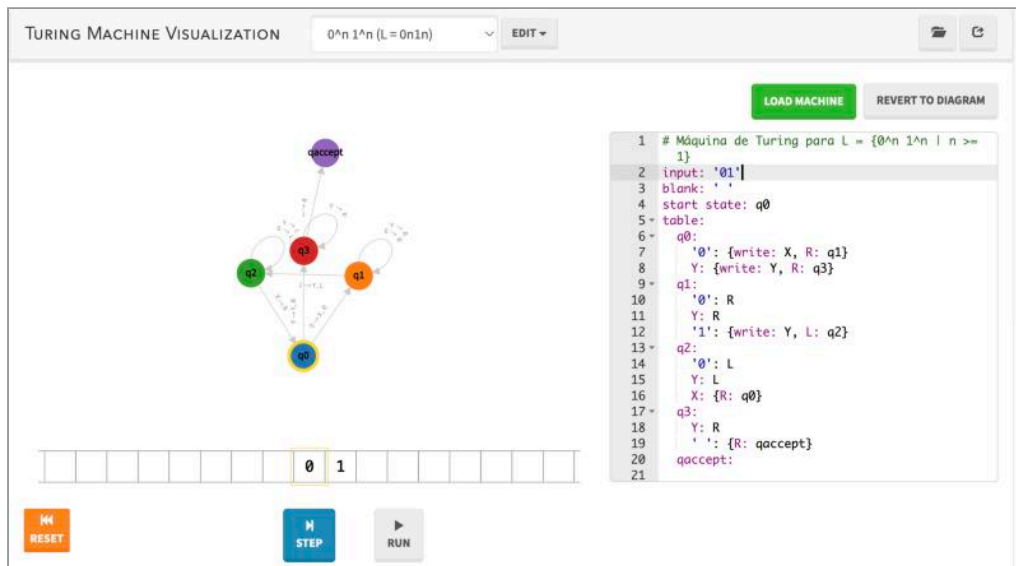
- ☒ **Etapa 1:** Respostas teóricas explicadas detalhadamente com rigor conceitual.
- ☒ **Etapa 2:** Definição formal da Máquina de Turing 0^n1^n e tabela de transições.
- ☒ **Etapa 3:** Registro de 3 testes executados com resultados e sequências de estados.
- ☒ **Etapa 4:** Reflexão fundamentada sobre a distinção entre problema difícil e impossível (Problema da Parada).
- ☒ **Anexo Prático:** Capturas de tela (prints) das 3 simulações rodando no simulador gráfico turingmachine.io estão coladas no anexo ao final deste relatório.

Anexo Prático: Capturas de Tela das Simulações

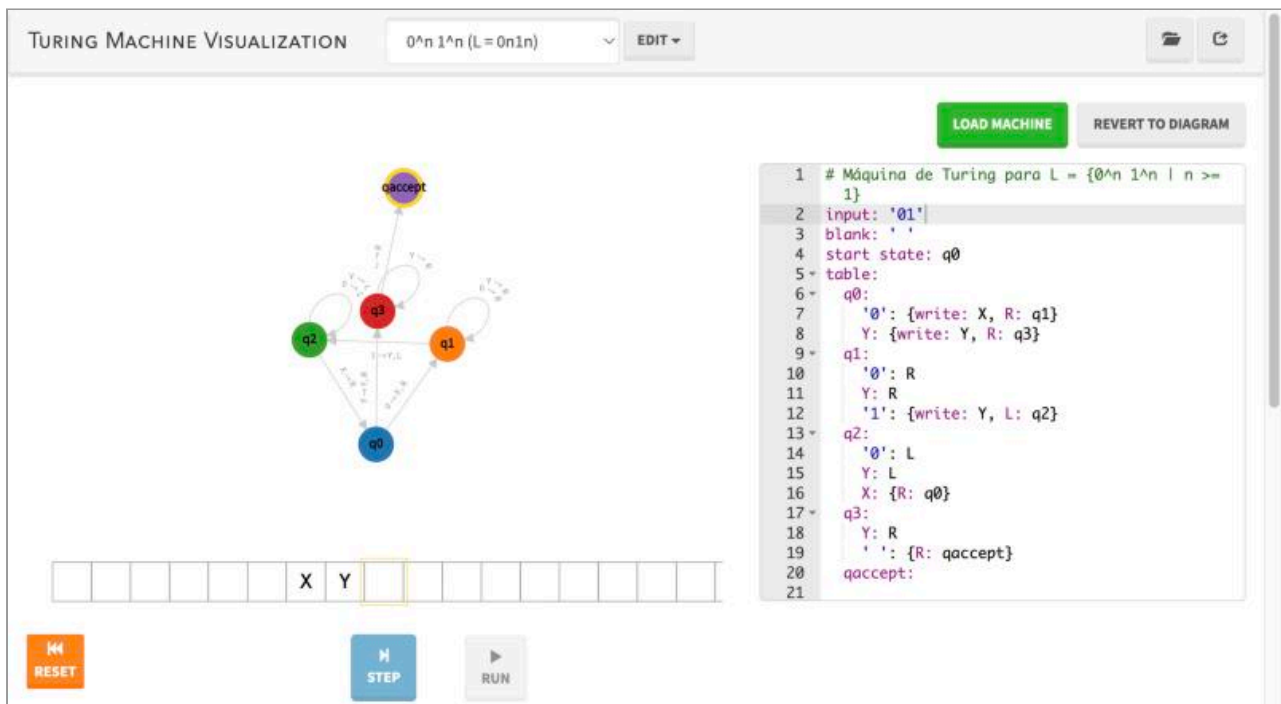
- **Simulador:** Turing Machine Visualization — <https://turingmachine.io>
- **Data de execução:** 2026-09-30
- **Máquina:** a do código de transições da Etapa 2 (syntaxe YAML do simulador, estado de parada `qaccept`); em cada teste mudou-se apenas a linha `input:`.

Nos três testes a máquina foi executada com o botão RUN até parar. O simulador não exibe mensagem textual de parada: a máquina parada é indicada pelos botões STEP e RUN desabilitados, com o estado corrente destacado no diagrama.

Teste 1: entrada 01

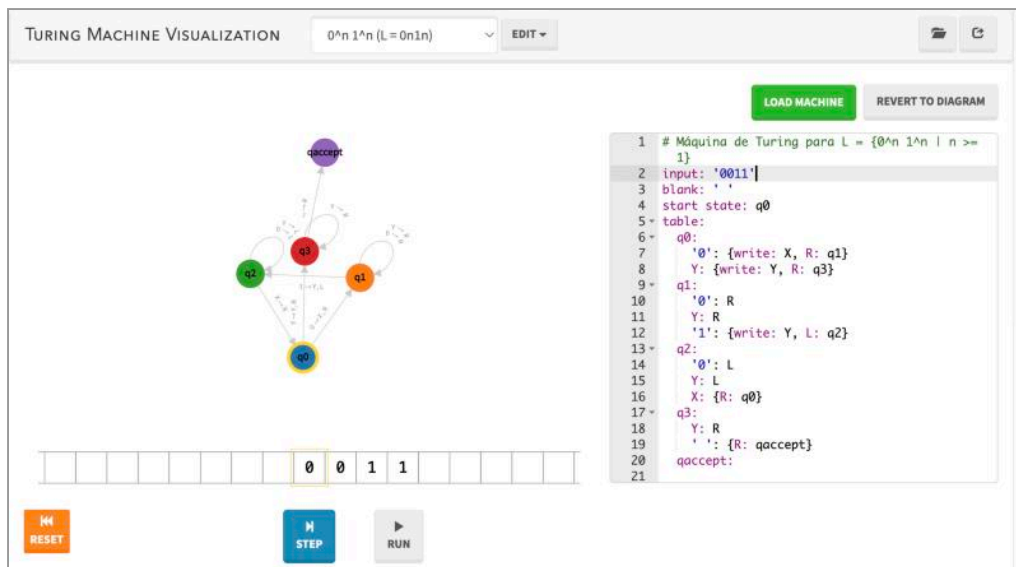


Teste 1 — estado inicial (antes do RUN): fita 01, estado q0

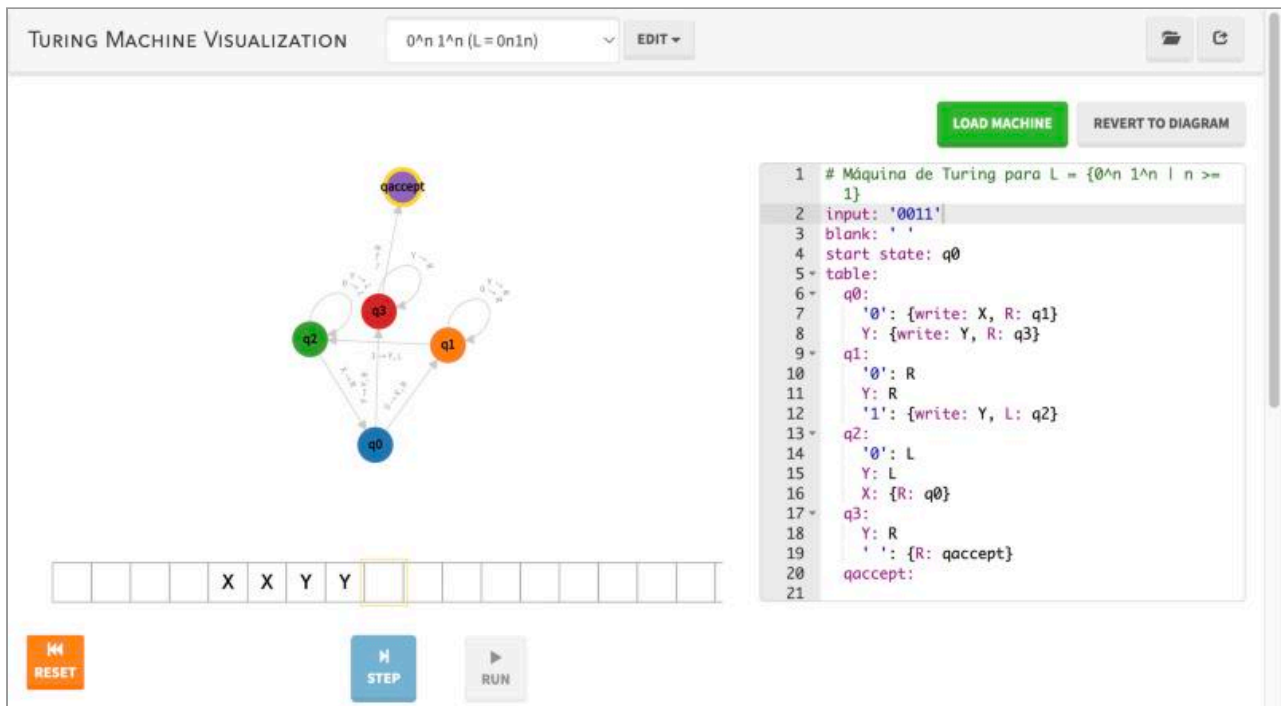


Teste 1 — entrada 01 — resultado: Aceito (estado final qaccept)

Teste 2: entrada 0011

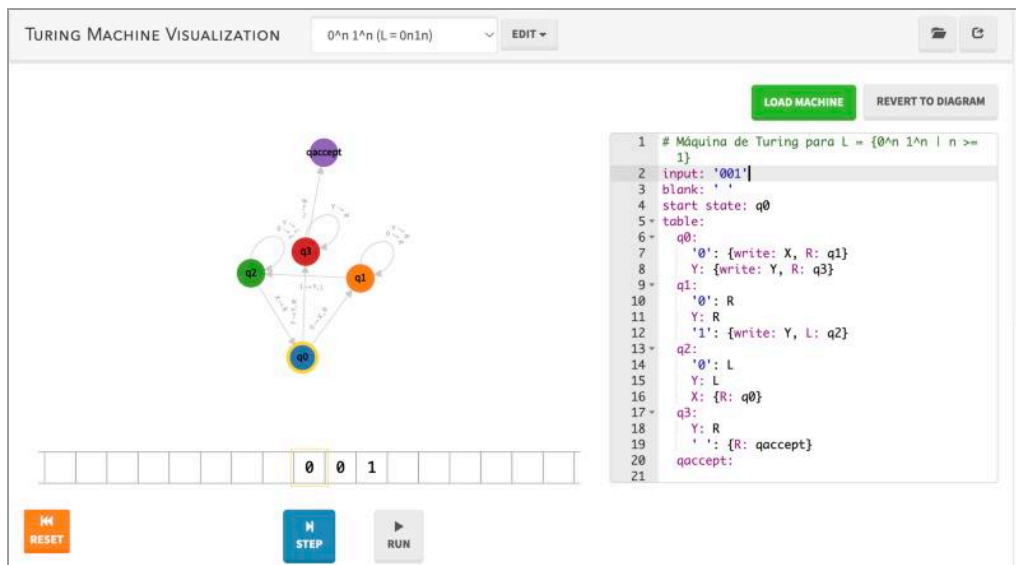


Teste 2 — estado inicial (antes do RUN): fita 0011, estado q0

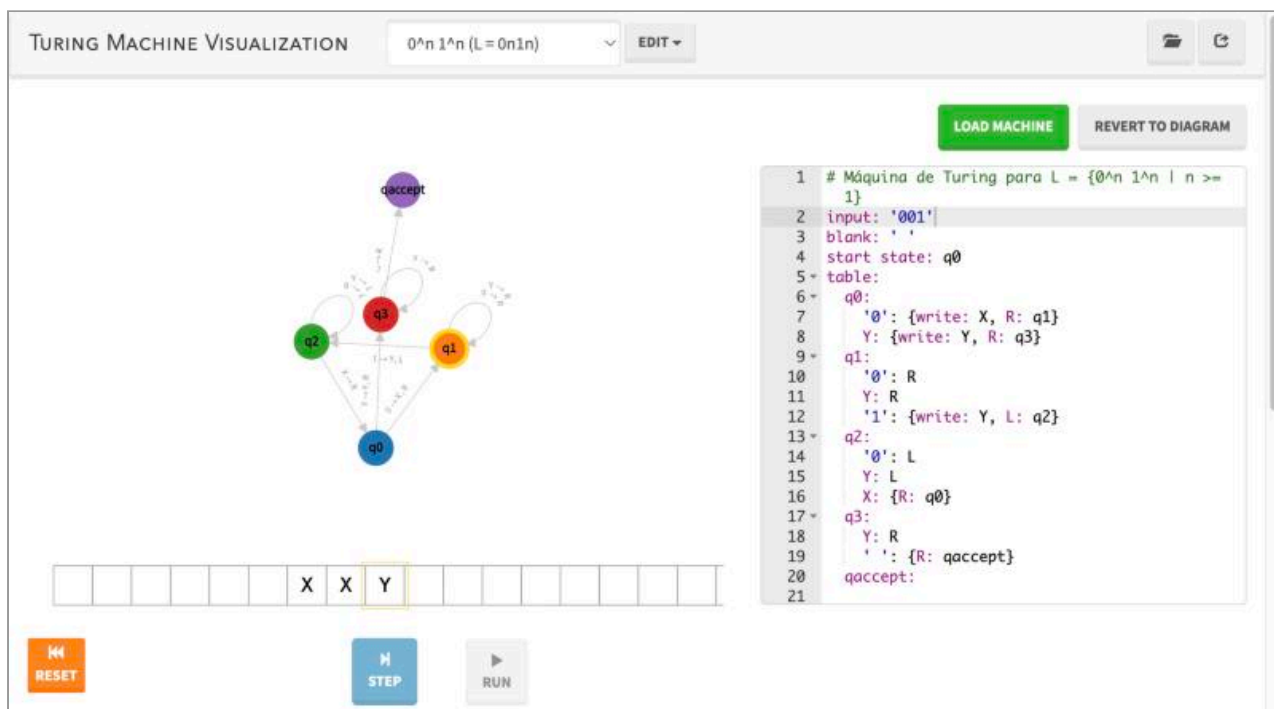


Teste 2 — entrada 0011 — resultado: Aceito (estado final qaccept)

Teste 3: entrada 001



Teste 3 — estado inicial (antes do RUN): fita 001, estado $q0$



Teste 3 — entrada 001 — resultado: Rejeitado (estado final $q1$, máquina travada)

Observação: a máquina travou em $q1$, sem chegar a q_{accept} , com STEP e RUN desabilitados. O quadro do cabeçote aparece desenhado sobre o último Y porque o simulador não anima o último deslocamento ao parar; o próximo símbolo a ser lido é o branco à direita do Y, para o qual $q1$ não tem transição.